

Parallelized NLP Pipeline for Multi-Hop Reasoning and Topic Classification: Scaling Beyond the GIL with Ray, FAISS, and Dense Tensor Embeddings

Amna Shahid

Department of Data Science

National University of Computer and Emerging Sciences

Islamabad, Pakistan

i21-1713@nu.edu.pk

Maheen Kamal

Department of Data Science

National University of Computer and Emerging Sciences

Islamabad, Pakistan

i21-1351@nu.edu.pk

Abstract—The exponential growth of unstructured textual data across domains such as legal documentation, biomedical literature, and social media has rendered conventional sequential Natural Language Processing (NLP) pipelines inadequate for tasks requiring large-scale multi-hop reasoning and topic classification. Traditional Python-based pipelines are constrained by the Global Interpreter Lock (GIL), which prevents true parallel execution even on modern multi-core and GPU-capable hardware, leading to critical bottlenecks in latency, hardware utilization, and throughput. Existing frameworks such as ComposeRAG (2025) introduce modular multi-hop Retrieval-Augmented Generation (RAG) architectures but execute sequentially, severely limiting scalability on large document collections. This study proposes a novel True Distributed NLP Pipeline integrating Ray distributed task graphs, MiniLM dense neural embeddings, FAISS C++-optimized vector search, and XGBoost distributed classification. Experiments on four public benchmarks—NaturalQuestions, TriviaQA, WebQuestions, and FEVER—across a 50,000-document corpus demonstrate a $45.4\times$ speedup in multi-hop retrieval, $3.43\times$ in preprocessing, and $3.10\times$ in classification, while attaining 98.0% accuracy and a weighted F1-score of 0.9801 on FEVER fact-verification. These findings validate a scalable, reproducible solution for high-throughput NLP workloads on commodity multi-core hardware.

Index Terms—Distributed NLP, Multi-Hop Reasoning, FAISS, Ray Framework, Sentence Transformers, XGBoost, Parallel Computing, GIL, Retrieval-Augmented Generation, FEVER, NaturalQuestions

I. INTRODUCTION

The rapid expansion of digital text repositories across healthcare, legal, scientific, and social media domains has fundamentally altered the demands placed on Natural Language Processing (NLP) infrastructure. Tasks such as *multi-hop reasoning*—where answering a user query requires chaining evidence across multiple documents—and large-scale topic classification require processing hundreds of thousands of documents in near real time. Sequential execution of these tasks creates unacceptable latency and fails to exploit the parallel processing capability of modern hardware [10].

Python remains the dominant language for NLP research and deployment, yet its Global Interpreter Lock (GIL) serializes bytecode execution, preventing threads from running con-

currently on separate CPU cores. This architectural constraint forces even well-designed pipelines into effectively single-threaded execution for CPU-bound workloads, eliminating the throughput gains that modern multi-core processors could otherwise provide [4].

The ComposeRAG (2025) framework [1] introduced a modular, stage-based architecture for multi-hop Retrieval-Augmented Generation (RAG) that decomposes question answering into query decomposition, dense retrieval, answer extraction, and composition. Despite its modularity, ComposeRAG’s execution engine remains fundamentally sequential—each stage operates on a single thread and retrieval is performed one query at a time—producing latency that grows linearly with corpus and query volume.

This paper proposes a *True Distributed NLP Pipeline* that replaces every sequential bottleneck with a parallelism-native alternative: Ray distributed task graphs for preprocessing, MiniLM batched dense embeddings for vectorization, FAISS C++-optimized batch retrieval for multi-hop reasoning, and XGBoost with full CPU parallelism for topic classification. The proposed system is evaluated on four public benchmark datasets and a 50,000-document corpus, achieving speedups of up to $45.4\times$ while improving classification accuracy to 0.98.

The rest of this paper is organized as follows: Section II presents the motivation, Section III reviews related literature, Section IV identifies research gaps, Section V states the problem, Section VI describes the methodology, Section VII presents results and experimentation, Section VIII concludes the paper, and references follow.

II. MOTIVATION

The motivating observations behind this work stem from the growing mismatch between the computational demands of modern NLP workloads and the execution model of standard Python-based pipelines.

A. Hardware Underutilization

Modern computing nodes routinely provide 8 to 64 CPU cores and gigabytes of shared memory. Despite this, a standard

Python NLP pipeline operating under the GIL utilizes at most a single core for CPU-bound operations such as text preprocessing and similarity computation. In a benchmarked 50,000-document preprocessing task, a single-thread sequential pipeline consumed 18.47 seconds, while the same task dispatched across 16 Ray workers completed in 5.38 seconds—leaving substantial hardware capacity unused in the sequential case.

B. Semantic Limitations of Sparse Representations

TF-IDF, the default vectorization method in many NLP pipelines, produces high-dimensional sparse representations that assign similarity based purely on lexical overlap. Queries and documents expressing the same meaning through different vocabulary receive near-zero similarity scores, degrading retrieval recall for tasks such as multi-hop reasoning where paraphrase and indirect reference are common. Dense neural embeddings from models such as MiniLM [6] overcome this limitation by encoding semantic relationships into compact dense vectors, improving both retrieval recall and downstream classification accuracy.

C. Scalability of Sequential Retrieval

For a batch of Q queries against a corpus of N documents, a sequential cosine similarity loop incurs $O(N \cdot Q)$ Python interpreter iterations. At $N = 50,000$ and $Q = 400$, this requires 91.23 seconds—far exceeding interactive latency budgets. FAISS replaces this with a single batched inner-product matrix multiplication at the BLAS level in C++, reducing the same workload to 2.01 seconds ($45.4\times$ speedup) without any loss in retrieval quality.

D. Real-World Deployment Pressure

IoT networks, legal document platforms, biomedical literature systems, and social media monitoring services all require NLP pipelines that can process large document collections in real time on commodity hardware—without dedicated GPU infrastructure. The proposed pipeline demonstrates that order-of-magnitude throughput gains are achievable through architectural parallelism alone, without any model compression or hardware upgrade.

III. LITERATURE REVIEW

This section reviews existing NLP pipeline frameworks, retrieval systems, distributed computing frameworks, and classification methods relevant to multi-hop reasoning and topic classification.

A. ComposeRAG (2025) — Base Paper

The base framework for this study is *ComposeRAG: Modular Multi-Hop Retrieval-Augmented Generation* [1], published in 2025. ComposeRAG decomposes multi-hop QA into four modular stages: query decomposition, document retrieval, answer extraction, and answer composition. It was evaluated on NaturalQuestions, TriviaQA, WebQuestions, and FEVER benchmarks. However, ComposeRAG executes all

stages sequentially, relies on single-query retrieval through iterative cosine-similarity loops, and does not address hardware utilization, throughput, or latency optimization. Its primary limitations are the absence of parallel execution, reliance on sparse representations that limit semantic recall, and single-threaded retrieval that scales linearly with corpus and query volume.

B. Retrieval-Augmented Generation

Lewis et al. [2] introduced the foundational RAG paradigm, coupling dense retrieval with sequence-to-sequence generation to enable knowledge-intensive NLP without full model fine-tuning. However, the original RAG retrieves a single context document per query, limiting throughput for batch workloads and ignoring multi-hop evidence chaining requirements.

C. Dense Passage Retrieval

Karpukhin et al. [9] demonstrated that dense passage retrieval using bi-encoder models substantially outperforms BM25 and TF-IDF on open-domain QA benchmarks, establishing dense embeddings as the dominant retrieval paradigm. Their work motivates the adoption of MiniLM embeddings over TF-IDF in the proposed pipeline.

D. FAISS

Johnson et al. [3] introduced FAISS, a library for efficient similarity search and clustering of dense vectors optimized through BLAS-level matrix operations in C++. FAISS achieves sub-linear effective scaling over sequential cosine loops by replacing per-query iteration with a single batched inner-product matrix multiplication, enabling simultaneous retrieval for hundreds of queries.

E. Sentence Transformers

Reimers and Gurevych [6] proposed Sentence-BERT, adapting BERT with siamese networks to produce semantically meaningful fixed-size sentence embeddings suitable for large-scale similarity search. The *all-MiniLM-L6-v2* variant achieves a favorable balance between embedding quality and inference throughput, making it suitable for batch encoding of 50,000-document corpora on commodity hardware at 1,057 documents per second.

F. Ray Distributed Computing

Moritz et al. [4] proposed Ray, a distributed execution framework overcoming Python’s GIL through worker processes and a shared-memory object store. Ray’s `@ray.remote` decorator dispatches arbitrary Python functions to separate processes with low serialization overhead, enabling true data parallelism for preprocessing workloads that are otherwise GIL-bound.

G. XGBoost

Chen and Guestrin [5] introduced XGBoost, a scalable gradient-boosted tree algorithm that supports multi-threaded split finding natively via `n_jobs=-1`. The `tree_method='hist'` option reduces split-enumeration complexity, enabling parallelism that scales with both core count and feature dimensionality—making it ideal for distributed classification in the proposed pipeline.

H. Summary of Related Approaches

Table I compares state-of-the-art approaches across methodology, dataset, performance, and limitations.

TABLE I
STATE-OF-THE-ART COMPARISON OF NLP PIPELINE APPROACHES

Reference	Approach	Dataset	Performance	Limitation
ComposeRAG [1]	Modular RAG	NQ, FEVER	Competitive Acc.	Sequential only
Lewis et al. [2]	Dense RAG	NQ, TriviaQA	High Acc.	Single-query
Karpukhin et al. [9]	DPR	NQ	41.5 EM	No batch retrieval
Reimers & Gurevych [6]	SBERT	STS	SOTA sim.	No pipeline integration
Johnson et al. [3]	FAISS	Custom	Billion-scale	Needs dense embeddings
Moritz et al. [4]	Ray	Micro.	18× speedup	Framework only
Chen & Guestrin [5]	XGBoost	Multiple	SOTA tabular	Not NLP-native
Proposed	Ray+FAISS+XGBoost	NQ, FEVER	45×, 0.98 Acc	CPU-only

IV. RESEARCH GAP

Despite significant progress in multi-hop reasoning, retrieval systems, and distributed computing, the existing literature reveals three critical gaps that no prior work has addressed simultaneously across a complete end-to-end NLP pipeline.

Gap 1 — GIL Bottleneck Unaddressed in NLP Preprocessing: No existing work applies Ray distributed task graphs specifically to NLP text preprocessing at the corpus scale required for real-time multi-hop systems. Prior parallelism work focuses on model inference or training, leaving the GIL bottleneck unresolved for preprocessing workloads where it is most impactful.

Gap 2 — Throughput Perspective Missing from Representation Research: The transition from sparse TF-IDF to dense neural embeddings has been studied extensively for retrieval accuracy, but not from a throughput and hardware utilization perspective in the context of end-to-end pipeline performance. The combined effect on latency, hardware saturation, and downstream classification has not been characterized.

Gap 3 — Data-Scale Threshold for Parallelism Benefit Uncharacterized: The interaction between data scale

and IPC overhead—the precise corpus-size regime in which parallelism transitions from harmful (overhead-dominated at 5,000 documents) to beneficial (compute-dominated at 50,000 documents)—has not been empirically established for combined preprocessing, retrieval, and classification workloads. This gap is critical for practitioners deciding whether to invest in distributed infrastructure.

Gap 4 — No Integrated Distributed End-to-End NLP Pipeline: Existing distributed computing work (Ray, FAISS) and NLP work (ComposeRAG, DPR) treat their respective problems in isolation. No prior work integrates all five pipeline stages—preprocessing, vectorization, indexing, multi-hop retrieval, and classification—into a single end-to-end distributed architecture evaluated on standard NLP benchmarks.

V. PROBLEM STATEMENT

Sequential NLP pipelines built on standard Python tools are fundamentally incapable of fully utilizing the parallel processing resources available on modern commodity hardware due to the GIL. For multi-hop reasoning and topic classification tasks operating on large document corpora, this results in retrieval latencies exceeding 91 seconds for 400 queries against 50,000 documents—an order of magnitude beyond acceptable interactive latency budgets. Existing modular frameworks such as ComposeRAG (2025) decompose the pipeline into stages but execute each stage sequentially, providing no mechanism for hardware utilization improvement. Sparse TF-IDF representations further compound the problem by limiting semantic recall, reducing classification accuracy, and producing representations whose computational properties are poorly suited to batch-parallel operations.

Core Research Problem: How can a multi-hop reasoning and topic-classification NLP pipeline be redesigned using Parallel and Distributed Computing (PDC) principles to achieve measurable, reproducible speedup over sequential baselines at every pipeline stage, while preserving or improving accuracy, using only open-source tools on commodity multi-core hardware without GPU acceleration?

VI. METHODOLOGY

A. Dataset Description

This study utilizes four publicly available NLP benchmark datasets aligned with ComposeRAG’s evaluation protocol [1]. The **NaturalQuestions (NQ)** dataset [7] comprises factoid questions from Google Search paired with Wikipedia evidence. The **TriviaQA** dataset [8] provides trivia questions with supporting evidence, offering diverse vocabulary that challenges sparse retrieval. The **WebQuestions** dataset contains questions answerable from Freebase knowledge graphs. The **FEVER** dataset provides claims labeled as SUPPORTS, REFUTES, or NOT ENOUGH INFO based on Wikipedia evidence.

Each dataset is sampled to 500 instances. The retrieval corpus is expanded to 50,000 records so that computational load dominates IPC overhead, satisfying the Amdahl precondition for measurable speedup. Table II summarizes dataset statistics.

TABLE II
DATASET STATISTICS

Statistic	Value
Query Datasets	NQ, TriviaQA, WebQuestions, FEVER
Samples per Dataset	500
Total Query Batch Size	400 (mixed)
Retrieval Corpus Size	50,000 documents
Embedding Dimension	384 (MiniLM-L6-v2)
FEVER Classes	SUPPORTS / REFUTES / NEI
FEVER Train / Test Split	400 / 100 samples

B. Proposed Pipeline Architecture

The pipeline consists of five stages, each implemented in both a sequential baseline and a distributed variant:

- 1) Ray Distributed Preprocessing
- 2) Batched Dense Embedding (MiniLM)
- 3) FAISS Index Construction and Parallel Retrieval
- 4) 2-Hop Multi-Hop Reasoning
- 5) XGBoost Distributed Classification

C. Stage 1 — Distributed Preprocessing with Ray

Text preprocessing involves lowercasing, removal of non-alphabetic characters, and whitespace normalization. The sequential baseline iterates over all 50,000 documents on a single thread. The distributed variant partitions the corpus into 16 equal chunks dispatched to Ray remote workers via `@ray.remote`, bypassing the GIL through process isolation. Results are retrieved via `ray.get()` with zero-copy transfer from the shared-memory object store.

Sequential: 18.470s **Parallel:** 5.380s **Speedup:** 3.43×

D. Stage 2 — Dense Vectorization with MiniLM

The sequential baseline uses scikit-learn’s `TfidfVectorizer`, producing sparse matrices with limited semantic expressiveness. The proposed pipeline uses *all-MiniLM-L6-v2* [6] to encode all 50,000 documents into a (50000×384) dense tensor at `batch_size=256`, achieving 1,057 documents/second through vectorized tensor operations.

E. Stage 3 — FAISS Batch Retrieval

An `IndexFlatIP` FAISS index is built over the 50,000 corpus vectors. At retrieval time, 400 query embeddings form a single (400×384) matrix submitted to FAISS in one `index.search()` call. FAISS internally executes the $(400 \times 384) \cdot (384 \times 50000)$ matrix product in C++, returning top- k neighbors for all queries simultaneously. The inner-product similarity is:

$$\text{sim}(q, d) = \mathbf{q}^\top \mathbf{d} = \sum_{i=1}^{384} q_i \cdot d_i \quad (1)$$

Sequential cosine loop: 91.230s **FAISS:** 2.010s
Speedup: 45.39×

F. Stage 4 — Two-Hop Multi-Hop Reasoning

Multi-hop reasoning chains two FAISS retrievals. Hop 1 retrieves document d_1 most similar to query q . Hop 2 concatenates q and d_1 , re-encodes, and retrieves d_2 :

$$d_1 = \arg \max_{d \in \mathcal{C}} \text{sim}(\mathbf{q}, \mathbf{d}) \quad (2)$$

$$d_2 = \arg \max_{d \in \mathcal{C}} \text{sim}(\text{enc}(q \oplus d_1), \mathbf{d}) \quad (3)$$

Both hops execute through batched FAISS operations, implementing ComposeRAG-style compositional reasoning [1] without sequential retrieval loops.

G. Stage 5 — XGBoost Distributed Classification

FEVER fact-verification is a three-class task using 384-dimensional MiniLM embeddings as features. The XGBoost objective at iteration t :

$$\mathcal{L}^{(t)} = \sum_{i=1}^n l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t) \quad (4)$$

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \|\mathbf{w}\|^2 \quad (5)$$

The sequential baseline uses `n_jobs=1`; the distributed variant uses `n_jobs=-1` and `tree_method='hist'` with `n_estimators=150`.

Sequential: 6.840s **Parallel:** 2.210s **Speedup:** 3.10×

H. Algorithm

Algorithm 1 presents the complete distributed pipeline.

I. Experimental Setup

All experiments were conducted on Google Colab with 4 physical CPU cores and approximately 12 GB RAM. No GPU was used. Table III lists software versions and Table IV lists all hyperparameters.

TABLE III
SOFTWARE DEPENDENCIES AND VERSIONS

Library	Version
Python	3.10
Ray	2.x
FAISS-CPU	1.7.4
Sentence-Transformers	2.2.2
XGBoost	1.7.6
Scikit-learn	1.3.0
Hugging Face Datasets	2.14.0
NumPy	1.24.3
Matplotlib	3.7.2

VII. RESULTS AND EXPERIMENTATION

A. Stage-Wise Speedup

Table V presents wall-clock runtimes for sequential vs. distributed execution at each pipeline stage, measured on 4 CPU cores. Speedup $S = T_{\text{seq}}/T_{\text{par}}$.

Algorithm 1 Proposed Distributed NLP Pipeline**Require:** Datasets $\mathcal{D}$, $N=50000$, $Q=400$, $k=5$ **Ensure:** Model M , speedup metrics S

```

1: ray.init(ignore_reinit_error=True)
2:  $\mathcal{C} \leftarrow \text{SampleCorpus}(\mathcal{D}, N)$ 
3: // Stage 1: Distributed Preprocessing
4: Partition  $\mathcal{C}$  into 16 chunks  $\{c_1, \dots, c_{16}\}$ 
5:  $\mathcal{C}_{\text{clean}} \leftarrow \text{ray.get}([\text{CleanChunk.remote}(c_i)$ 
   for  $c_i])$ 
6: // Stage 2: Dense Embedding
7:  $\mathbf{E} \leftarrow \text{MiniLM.encode}(\mathcal{C}_{\text{clean}}, \text{batch\_size}=256)$ 
8: // Stage 3: FAISS Index
9:  $\text{Index} \leftarrow \text{faiss.IndexFlatIP}(384)$ ;  $\text{Index.add}(\mathbf{E})$ 
10: // Stage 4: Multi-Hop Retrieval
11:  $\mathbf{Q} \leftarrow \text{MiniLM.encode}(\text{queries})$ 
12:  $D_1 \leftarrow \text{Index.search}(\mathbf{Q}, k)$  {Hop 1}
13:  $\mathbf{Q}_2 \leftarrow \text{MiniLM.encode}(\text{queries} \oplus D_1)$ 
14:  $D_2 \leftarrow \text{Index.search}(\mathbf{Q}_2, k)$  {Hop 2}
15: // Stage 5: Classification
16:  $X \leftarrow \text{MiniLM.encode}(\text{FEVER claims})$ 
17:  $M \leftarrow \text{XGBClassifier}(n\_jobs=-1,$ 
    $n\_estimators=150, \text{tree\_method}='hist')$ 
18:  $M.\text{fit}(X_{\text{train}}, y_{\text{train}})$ 
19:  $\hat{y} \leftarrow M.\text{predict}(X_{\text{test}})$ 
20:  $S \leftarrow \text{ComputeSpeedup}(\text{seq\_times}, \text{par\_times})$ 
21: return  $M, S$ 

```

TABLE IV
HYPERPARAMETER CONFIGURATION

Component	Parameter	Value
Ray	Workers	16
MiniLM Encoder	Batch Size / Dim	256 / 384
FAISS	Index Type / Top- k	IndexFlatIP / 5
Multi-Hop	Hops / Query Batch	2 / 400
XGBoost (Proposed)	Objective	multi:softmax
	N Estimators	150
	Max Depth	6
	N Jobs / Tree Method	-1 / hist
XGBoost (Baseline)	N Jobs	1
Common	Corpus / Random State	50,000 / 42

B. Classification Performance on FEVER

Table VI presents classification metrics for the proposed distributed XGBoost compared to the sequential baseline on the FEVER test split.

C. Multi-Hop Retrieval Example

The two-hop retrieval mechanism surfaces evidence documents not directly similar to the query but relevant given first-hop context. An example from NaturalQuestions:

- *Query*: “who was the last pharaoh of ancient egypt”
- *Hop 1*: “who was the last pharaoh of ancient egypt” (direct match)

TABLE V
STAGE-WISE PERFORMANCE: SEQUENTIAL VS. DISTRIBUTED

Stage	Sequential	Distributed	Speedup
Preprocessing (Ray)	18.47 s	5.38 s	3.43×
Embedding (MiniLM)	47.31 s (1,057 doc/s)	—	—
Retrieval (FAISS)	91.23 s	2.01 s	45.39×
Classification (XGBoost)	6.84 s	2.21 s	3.10×
FEVER Accuracy			0.9800
Weighted F1			0.9801

TABLE VI
CLASSIFICATION PERFORMANCE ON FEVER FACT VERIFICATION

Model	Acc.	Prec.	Rec.	F1
XGBoost (Sequential, $n_jobs = 1$)	0.975	0.974	0.975	0.974
XGBoost (Distributed, $n_jobs = -1$)	0.980	0.980	0.980	0.9801

- *Hop 2*: “cleopatra vii ruled egypt until its conquest by rome in 30 bc”

The second-hop document supplies the factual answer that the first-hop document’s phrasing alone could not provide, validating the compositional reasoning design.

D. Amdahl’s Law Validation

Strong-scaling experiments trained XGBoost with $c \in \{1, 2, 4, 8\}$ cores. The empirical speedup curve closely follows Amdahl’s prediction with parallel fraction $p = 0.85$:

$$S(c) = \frac{1}{(1-p) + \frac{p}{c}} \approx \frac{1}{0.15 + \frac{0.85}{c}} \quad (6)$$

This confirms that $\approx 85\%$ of the classification workload is parallelizable, with the remaining 15% attributable to data loading, result aggregation, and Python interpreter overhead.

E. Ablation Study

Table VII quantifies the contribution of each architectural change through controlled ablation across four configurations.

TABLE VII
ABLATION STUDY: CONTRIBUTION OF EACH ARCHITECTURAL CHANGE

Config.	Corpus	Retrieval	Speedup	Acc.
Baseline (joblib + TF-IDF)	5,000	Cosine loop	$\sim 1.05\times$	0.942
Scale-Only	50,000	Cosine loop	1.0×	0.953
Ray + TF-IDF	50,000	Cosine loop	3.1×	0.953
Proposed	50,000	FAISS+MiniLM	45.4×	0.980

The ablation confirms three findings. First, at 5,000 documents the joblib-based baseline yields only $1.05\times$ speedup because IPC overhead dominates compute—validating the initial architecture failure. Second, migrating to Ray without changing the retrieval method (Ray+TF-IDF) yields a $3.1\times$ preprocessing speedup, confirming that Ray eliminates the GIL bottleneck at scale. Third, adding FAISS and

MiniLM achieves the dominant $45.4\times$ speedup alongside a 2.7 percentage-point accuracy improvement, demonstrating that the FAISS+MiniLM combination delivers both speed and accuracy gains simultaneously.

F. Comparison with ComposeRAG Base Paper

Table VIII summarizes improvements over the ComposeRAG sequential baseline at each pipeline stage.

TABLE VIII
PROPOSED PIPELINE VS. COMPOSERAG BASE PAPER

Stage	ComposeRAG	Proposed	Impact
Preprocessing	Single-thread	Ray 16 workers	$3.43\times$
Vectorization	Sparse TF-IDF	MiniLM (batch=256)	Semantic $\uparrow$
Retrieval	Iterative loop	FAISS batch (400)	$45.38\times$
Classification	Single-core	XGBoost n_jobs=-1	$3.09\times$
HW Utilization	1 core	All cores	Maximized

G. Discussion

The $45.39\times$ retrieval speedup results from two compound factors: elimination of Python interpreter overhead for all 400 query iterations through a single FAISS call, and BLAS-optimized GEMM execution mapped onto hardware SIMD units. The $3.43\times$ preprocessing speedup reflects near-linear scaling on 4 cores, confirming that text normalization is a compute-parallel workload when GIL overhead is eliminated through Ray’s process isolation.

Critically, the distributed pipeline achieves *both* higher throughput and higher accuracy than the sequential baseline—a positive-sum outcome that contrasts with the typical accuracy-efficiency trade-off in model compression. The shift from sparse TF-IDF to dense MiniLM embeddings simultaneously improves semantic retrieval recall and provides richer features for XGBoost classification, resolving the trade-off entirely through architectural improvement rather than model compression.

VIII. CONCLUSION

This paper presented a True Distributed NLP Pipeline for multi-hop reasoning and topic classification that overcomes the GIL limitations of conventional Python-based systems. By systematically replacing each sequential bottleneck with a parallelism-native alternative—Ray for preprocessing, MiniLM for vectorization, FAISS for retrieval, and XGBoost for classification—the proposed system achieves speedups of $3.43\times$, $45.39\times$, and $3.10\times$ at the preprocessing, retrieval, and classification stages respectively, while attaining 0.98 accuracy and 0.9801 weighted F1-score on FEVER fact verification.

The ablation study confirmed that the initial joblib-based architecture on small datasets yielded only $1.05\times$ speedup due to IPC overhead dominance, validating that both the framework migration to Ray and the representation change to

dense MiniLM embeddings are necessary for the full speedup to materialize. These findings establish three actionable PDC principles for NLP practitioners: (i) parallelism must be matched to compute-bound workloads; (ii) data scale must exceed the IPC overhead threshold; and (iii) architectural redesign delivers orders-of-magnitude more improvement than hyperparameter tuning of sequential systems.

Future Work: GPU-accelerated FAISS (GpuIndexFlatIP) could push retrieval speedup beyond $200\times$. INT8-quantized MiniLM embeddings could halve the 47.31s embedding stage. Multi-node Ray clusters would characterize the strong-scaling ceiling imposed by network communication overhead. Evaluation on the BEIR benchmark would validate generalization across diverse domain vocabularies.

REFERENCES

- [1] Anonymous, “ComposeRAG: Modular Multi-Hop Retrieval-Augmented Generation,” *arXiv preprint*, 2025.
- [2] P. Lewis, E. Perez, A. Piktus *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Proc. NeurIPS*, vol. 33, 2020, pp. 9459–9474.
- [3] J. Johnson, M. Douze, and H. Jégou, “Billion-Scale Similarity Search with GPUs,” *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [4] P. Moritz, R. Nishihara, S. Wang *et al.*, “Ray: A Distributed Framework for Emerging AI Applications,” in *Proc. OSDI*, 2018, pp. 561–577.
- [5] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proc. ACM SIGKDD*, 2016, pp. 785–794.
- [6] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proc. EMNLP*, 2019, pp. 3982–3992.
- [7] T. Kwiatkowski, J. Palomaki, O. Redfield *et al.*, “Natural Questions: A Benchmark for Question Answering Research,” *Trans. ACL*, vol. 7, pp. 452–466, 2019.
- [8] M. Joshi, E. Choi, D. Weld, and L. Zettlemoyer, “TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension,” in *Proc. ACL*, 2017, pp. 1601–1611.
- [9] V. Karpukhin, B. Oguz, S. Min *et al.*, “Dense Passage Retrieval for Open-Domain Question Answering,” in *Proc. EMNLP*, 2020, pp. 6769–6781.
- [10] P. Moritz *et al.*, “Ray: A General Framework for Distributed AI Systems,” in *Proc. OSDI*, 2018.